You can also control backtracking by using a third form of repetition. We've already seen greedy repetition, such as EXPRESSION+, and lazy repetition, EXPRESSION+?. The third form is called *possessive*. You code it using a plus sign after the repetition character, as in EXPRESSION++. It behaves just like greedy repetition, consuming as much of the string as it can. But once consumed, that part of the string can never be reexamined by the pattern—the regular expression engine can't backtrack past a possessive qualifier. This means we could also write our code as this:

```
re = /(X++)(?!O)/
re.match("test XXXY")[0]       # => "XXX"
re.match?("test XXXO")         # => false
re.match("test XXXO XXXXY")[0] # => "XXXX"
```

Backreferences and Named Matches

Within a pattern, the sequences \n (where *n* is a number), \k'n', and \k<n> all refer to the n[th] captured subpattern. Thus, the expression /(...)\1/ matches six characters with the first three characters being the same as the last three.

Rather than refer to matches by their number, you can give them names and then refer to those names. A subpattern is named using either of the syntaxes (?<name>...) or (?'name'...). You then refer to these named captures using either \k<name> or \k'name'.

For example, the following shows different ways of matching a time range (in the form hh:mm-hh:mm) where the hour is the same on both sides of the range:

tut_regexp/named_backreference_1.rb

```
same   = "12:15-12:45"
differ = "12:45-13:15"

 # use numbered backreference
same   =~ /(\d\d):\d\d-\1:\d\d/              # => 0
differ =~ /(\d\d):\d\d-\1:\d\d/              # => nil

 # use named backreference
same   =~ /(?<hour>\d\d):\d\d-\k<hour>:\d\d/  # => 0
differ =~ /(?<hour>\d\d):\d\d-\k<hour>:\d\d/  # => nil
```

Negative backreference numbers count backward from the place they're used, so they are relative, not absolute, numbers. The following pattern matches four-letter palindromes (words that read the same forward and backward):

tut_regexp/named_backreference_2.rb

```
"abab" =~ /(.)(.)\k<-1>\k<-2>/  # => nil
"abba" =~ /(.)(.)\k<-1>\k<-2>/  # => 0
```

You can invoke a named subpattern using \g<name> or \g<number>. Note that this reexecutes the match in the subpattern, in contrast to \k<name>, which matches whatever is matched by the subpattern:

tut_regexp/named_backreference_3.rb

```
re = /(?<color>red|green|blue) \w+ \g<color> \w+/
re =~ "red sun blue moon"   # => 0
re =~ "red sun white moon"  # => nil
```

You can use \g recursively, invoking a pattern within itself. The following code matches a string in which braces are properly nested: